

RECUPERACIÓN DE CONTRASEÑAS

1. Descripción General del Flujo

El sistema implementa DOS flujos independientes de recuperación de contraseña, ambos diseñados con principios de seguridad por defecto y mensajes no reveladores (no indican si un número existe).

Flujo Legacy (Token por enlace)	Flujo OTP (Código de 6 dígitos)
POST /auth/recover POST /auth/reset-password <i>Token: 32 bytes aleatorios (hex, 64 chars)</i> <i>Expiración: 60 minutos</i>	POST /auth/request-otp POST /auth/verify-otp POST /auth/reset-password-otp <i>OTP: 6 dígitos + max 3 intentos</i> <i>Expiración OTP: 5 min Reset token: 10 min</i>

2. Flujo Legacy — Token por Enlace

2.1 Solicitud de Recuperación (POST /auth/recover)

El usuario proporciona su número de teléfono (10 dígitos). El endpoint SIEMPRE responde con el mismo mensaje genérico, sin importar si el número existe o no. Esta técnica se llama "respuesta ambigua" o non-revealing response.

Mensaje accesible implementado:

```
"Si el número está registrado, recibirás instrucciones para restablecer tu contraseña."
```

Proceso interno (recoveryController.js):

1. Validar formato del teléfono (10 dígitos exactos).
2. Si el usuario existe: marcar tokens anteriores como usados (used = TRUE).
3. Generar token seguro: `crypto.randomBytes(32).toString('hex')` — 256 bits de entropía.
4. Hashear el token con SHA-256 antes de guardarlo en BD.
5. Almacenar token_hash en tabla password_resets con expiración de 60 minutos.
6. En desarrollo: imprimir el token en consola para pruebas.
7. Responder siempre con el mismo mensaje genérico (200 OK).

Código fuente — recoveryController.js:

```
function hashToken(plain) {
  return crypto.createHash('sha256').update(plain, 'utf8').digest('hex');
}

function generateResetToken() {
  return crypto.randomBytes(32).toString('hex');
}

// Al solicitar recuperación:
const plain = generateResetToken();
const tokenHash = hashToken(plain);
const expiresAt = new Date(Date.now() + 60 * 60 * 1000); // 1 hora
```

2.2 Cambio de Contraseña (POST /auth/reset-password)

El usuario envía: teléfono, token recibido, y nueva contraseña.

Validaciones de seguridad:

- Teléfono: string de 10 dígitos exactos.
- Token: string no vacío.
- Nueva contraseña: mínimo 6 caracteres.

Proceso interno:

8. Hashear el token recibido con SHA-256.
9. Buscar en BD: phone + token_hash + used = FALSE + expires_at > NOW().
10. Si no existe: responder 'Token inválido o expirado' (sin revelar el motivo exacto).
11. Verificar que el usuario aún existe en la BD.
12. Hashear nueva contraseña con bcrypt (salt factor 10).
13. Actualizar password_hash del usuario.
14. Marcar el token como usado (used = TRUE) — previene replay.
15. Revocar TODAS las sesiones activas del usuario (revokeAllUserSessions).

3. Flujo OTP — Código de 6 Dígitos

3.1 Solicitud de OTP (POST /auth/request-otp)

El usuario proporciona su número de teléfono. Se genera un OTP aleatorio de 6 dígitos usando `crypto.randomInt()` (criptográficamente seguro).

Seguridades del OTP (`otpRecoveryService.js`):

- No se almacena en claro: se hashea con SHA-256 + pepper + teléfono vinculado.
- Al pedir un nuevo OTP se eliminan los anteriores (previene acumulación).
- Expiración corta: 5 minutos.
- Máximo 3 intentos fallidos antes de invalidar el código.
- Respuesta genérica: el sistema no revela si el número existe.

Código fuente — función `hashOtp`:

```
function hashOtp(phone, sixDigitCode) {
  const pepper = process.env.OTP_PEPPER || 'change-me-in-production';
  return crypto
    .createHash('sha256')
    .update(`${pepper}:${phone}:${sixDigitCode}`, 'utf8')
    .digest('hex');
}

function generateSixDigitOtp() {
  return String(crypto.randomInt(0, 1_000_000)).padStart(6, '0');
}
```

3.2 Verificación del OTP (POST /auth/verify-otp)

El usuario envía teléfono + código de 6 dígitos.

Verificación segura con `timingSafeEqual`:

```
function timingSafeEqualHex(a, b) {
  const ba = Buffer.from(a, 'hex');
  const bb = Buffer.from(b, 'hex');
  if (ba.length !== bb.length) return false;
  return crypto.timingSafeEqual(ba, bb);
}
```

timingSafeEqual previene timing attacks: siempre compara en tiempo constante, sin revelar cuántos dígitos son correctos.

Si el OTP es válido:

16. Eliminar el OTP de la BD (previene reutilización).
17. Invalidar todos los reset tokens anteriores del usuario.
18. Generar un resetToken de 32 bytes aleatorios.
19. Almacenar hash del resetToken con expiración de 10 minutos.
20. Retornar el resetToken al cliente para el siguiente paso.

3.3 Cambio de Contraseña con OTP (POST /auth/reset-password-otp)

El usuario envía: teléfono, resetToken obtenido en el paso anterior, nueva contraseña.

Proceso:

21. Hashear el resetToken con SHA-256.
22. Verificar en tabla otp_reset_tokens: user_id + hash + used = FALSE + expires_at > NOW().
23. Hashear nueva contraseña con bcrypt (salt 10).
24. Actualizar password_hash.
25. Marcar token como usado.
26. Revocar todas las sesiones activas del usuario.

4. Esquema de Base de Datos

Tabla	password_resets
Campos	id, phone, token_hash, expires_at, used, created_at
Índices	idx_password_resets_phone_used, idx_password_resets_expires
Uso	Flujo legacy de recuperación por enlace/token

Tabla	otp_codes
--------------	-----------

Campos	id, user_id (FK), phone, code_hash, expires_at, attempts, created_at
Índices	idx_otp_codes_user_id, idx_otp_codes_expires
Uso	Almacena el OTP hasheado con contador de intentos

Tabla	otp_reset_tokens
Campos	id, user_id (FK), token_hash, expires_at, used, created_at
Índices	idx_otp_reset_tokens_user_id, idx_otp_reset_tokens_hash_used
Uso	Token de un solo uso emitido tras verificar OTP correctamente

5. Rutas Implementadas

Método	Ruta	Descripción
POST	/auth/recover	Solicita recuperación — genera token legacy (60 min)
POST	/auth/reset-password	Valida token legacy y cambia contraseña
POST	/auth/request-otp	Genera OTP de 6 dígitos (5 min, max 3 intentos)
POST	/auth/verify-otp	Verifica OTP y emite resetToken (10 min, 1 uso)
POST	/auth/reset-password-otp	Valida resetToken y cambia contraseña

6. Tokens Seguros y Expiración

6.1 Generación de Tokens

Token legacy	crypto.randomBytes(32).toString('hex') — 64 caracteres hex, 256 bits de entropía
OTP	crypto.randomInt(0, 1_000_000) — criptográficamente seguro, 6 dígitos
Reset OTP token	crypto.randomBytes(32).toString('hex') — mismo nivel que legacy
OTP pepper	Variable de entorno OTP_PEPPER vinculada al teléfono — previene ataques offline a OTPs robados

6.2 Tiempos de Expiración Justificados

OTP (6 dígitos)	5 minutos — ventana muy corta porque el espacio de valores es pequeño (1M combinaciones)
Reset token OTP	10 minutos — emitido tras verificar OTP; suficiente para completar el formulario
Token legacy	60 minutos — compatible con flujos de email/SMS más lentos; mayor entropía compensa
Invalidación inmediata	Todos los tokens se marcan used=TRUE al ser usados o al pedir uno nuevo

7. Mensajes Accesibles y No Reveladores

Los mensajes de respuesta cumplen dos criterios simultáneos: son claros para el usuario legítimo y no revelan información al atacante.

Situación	Mensaje al usuario	Por qué es seguro
Solicitar recuperación (usuario existe o no)	Si el número está registrado, recibirás instrucciones para restablecer tu contraseña.	No confirma ni niega la existencia del número
Solicitar OTP (usuario existe o no)	Si el número existe, se enviará un código	Idéntico para cualquier teléfono
OTP incorrecto / expirado / agotado	Código inválido o expirado	No indica cuántos intentos quedan ni qué falló
Token legacy inválido o expirado	Token inválido o expirado.	No distingue entre token incorrecto y expirado
Reset token OTP inválido	Token inválido o expirado.	Consistente con el mensaje legacy
Cambio exitoso (OTP)	Contraseña actualizada correctamente	Mensaje positivo, accesible, sin datos extra
Cambio exitoso (legacy)	Contraseña actualizada. Inicia sesión de nuevo.	Indica la acción requerida al usuario

8. Evidencia Funcional — Endpoints

8.1 Flujo Legacy

Paso 1: Solicitar recuperación

```
POST /auth/recover
Content-Type: application/json

{ "phone": "1234567890" }
```

```
// Respuesta (200) – misma sin importar si el usuario existe:
{
  "msg": "Si el número está registrado, recibirás instrucciones para
         restablecer tu contraseña."
}
```

```
// En desarrollo, consola muestra:
// [password reset demo] phone=1234567890 token=a3f8b2...c7d1e4
```

Paso 2: Cambiar contraseña con token

```
POST /auth/reset-password

{
  "phone": "1234567890",
  "token": "a3f8b2...c7d1e4",
  "password": "NuevaPassSegura123"
}
```

```
// Respuesta exitosa (200):
{ "msg": "Contraseña actualizada. Inicia sesión de nuevo." }
```

```
// Token inválido/expirado (400):
{ "msg": "Token inválido o expirado." }
```

8.2 Flujo OTP

Paso 1: Solicitar OTP

```
POST /auth/request-otp
{ "phone": "1234567890" }
```

```
// Respuesta (200):
{ "msg": "Si el número existe, se enviará un código", "otp": "482951" }
// Nota: otp solo aparece en NODE_ENV !== 'production'
```

Paso 2: Verificar OTP

```
POST /auth/verify-otp
{ "phone": "1234567890", "code": "482951" }
```

```
// Respuesta exitosa (200):
{ "ok": true, "resetToken": "b7e2a1...f8c3d0" }
```

```
// OTP incorrecto (400):  
{ "ok": false, "msg": "Código inválido o expirado" }
```

Paso 3: Cambiar contraseña con resetToken

```
POST /auth/reset-password-otp  
{  
  "phone": "1234567890",  
  "resetToken": "b7e2a1...f8c3d0",  
  "newPassword": "NuevaPassSegura123"  
}
```

```
// Respuesta exitosa (200):  
{ "ok": true, "msg": "Contraseña actualizada correctamente" }
```

9. Validaciones en las Rutas

Cada endpoint usa express-validator antes de llegar al controlador:

- phone: isLength({ min: 10, max: 10 }) — rechaza teléfonos mal formados.
- token / resetToken: not().isEmpty() — campo obligatorio.
- password / newPassword: isLength({ min: 6 }) — contraseña mínima.
- code: matches(/^d{6}\$/) — exactamente 6 dígitos, rechaza códigos con letras.

Errores de validación retornan 400 con array de errores descriptivos, nunca revelan estado interno.

10. Qué se Puede Añadir para Evidencia Adicional

Las siguientes mejoras pueden implementarse sobre la base actual para enriquecer la evidencia:

10.1 Rate Limiting

Limitar intentos por IP y por teléfono usando express-rate-limit. Ejemplo:

```
npm install express-rate-limit
```



```
const rateLimit = require('express-rate-limit');
const otpLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 min
  max: 5, // 5 solicitudes por IP
  message: { msg: 'Demasiados intentos. Intenta en 15 minutos.' }
});
router.post('/request-otp', otpLimiter, [...], requestOtpHandler);
```

10.2 Pruebas Automatizadas Jest

El proyecto ya tiene Jest configurado (tests/session.test.js). Se pueden añadir:

- Test de solicitud con teléfono inexistente → mismo mensaje que existente.
- Test de token expirado → 400 con mensaje genérico.
- Test de OTP con más de 3 intentos → invalidación automática.
- Test de reutilización de token → debe rechazar con used = TRUE.

10.3 Logging de Auditoría

El proyecto ya tiene auditoriaController.js. Se puede registrar:

- Cada solicitud de recuperación (sin el token, solo el evento).
- Cada intento de verificación fallido con timestamp e IP.
- Cada cambio de contraseña exitoso.

10.4 Notificación Simulada

Crear un módulo notifications.js que simule el envío:

```
async function sendRecoveryToken(phone, token) {
  if (process.env.NODE_ENV !== 'production') {
    console.log(`[SMS simulado] Para ${phone}: token=${token}`);
    return;
  }
  // En producción: integrar Twilio, AWS SNS, etc.
}
```

11. Resumen de Características de Seguridad Implementadas

Característica	Estado	Archivo
----------------	--------	---------

Tokens criptográficamente seguros (32 bytes)	Implementado	recoveryController.js, otpRecoveryService.js
Expiración de tokens (5, 10 y 60 min)	Implementado	otpRecoveryService.js, recoveryController.js
Hashing de tokens con SHA-256	Implementado	Ambos flujos
OTP hasheado con pepper + teléfono vinculado	Implementado	otpRecoveryService.js
timingSafeEqual para comparar OTPs	Implementado	otpRecoveryService.js
Límite de intentos OTP (máx. 3)	Implementado	otpRecoveryService.js
Tokens de un solo uso (used = TRUE)	Implementado	Ambas tablas
Invalidación de tokens anteriores al pedir uno nuevo	Implementado	Ambos flujos
Revocación de sesiones tras cambio de contraseña	Implementado	sessionService.js
Mensajes no reveladores	Implementado	Ambos controladores
Validación de entrada (express-validator)	Implementado	authRoutes.js
Rate limiting	Recomendado	Por añadir con express-rate-limit
Pruebas automatizadas de recuperación	Recomendado	Por añadir en tests/